

Health-Checked Migration

Proof of Concept

Pre-Validation of Preferred Servers Before QUIC Connection Migration

PoC Documentation • July 2026

This document describes the Health-Checked Migration proof of concept, which adds pre-validation of the preferred server before advertising `preferred_address` in the QUIC handshake. Without health checking, a primary server unconditionally advertises a preferred address; if the preferred server is down, the client experiences a `PATH_CHALLENGE` timeout (100-600ms). Our PoC implements two health check modes -- **TCP Probe** and **Redis Heartbeat** -- that allow the primary server to detect preferred server failures and gracefully fall back to serving clients directly. Health probes use a **dedicated port 9998**, fully separated from the state transfer port 9999, preventing probe interference with migration state channels. The result is zero user-visible errors during preferred server outages, with negligible added latency (<0.2ms per connection).

Contents

1. Overview
 - 1.1 The Problem
 - 1.2 The Solution
2. Architecture
 - 2.1 Testbed
 - 2.2 Port Separation: Health (9998) vs State (9999)
 - 2.3 Health Check Modes
 - 2.4 Migration Flow Diagram (Success)
 - 2.5 Failure Scenario Diagram
3. Implementation Details
 - 3.1 Binaries and Configuration
 - 3.2 Background Health Thread
 - 3.3 Heartbeat Thread Behavior
 - 3.4 Redis Heartbeat Protocol
 - 3.5 Conditional preferred_address
 - 3.6 HTML Status Page
4. Security Considerations
5. Metrics and Monitoring
6. How to Run
7. Test Scenarios
8. Expected Results
9. Comparison Table

1. Overview

1.1 The Problem

In our current QUIC server-side migration implementation, the primary server **unconditionally** advertises a `preferred_address` transport parameter during the TLS handshake. This tells the client to migrate its connection to the preferred server's address. However, if the preferred server is down or unreachable at the time of migration, the client sends a `PATH_CHALLENGE` to an unresponsive host and experiences a timeout.

Problem: The client has no way to know the preferred server is down until `PATH_CHALLENGE` times out (100-600ms depending on implementation). This causes user-visible latency spikes and potential connection failures.

1.2 The Solution

The solution is to add a **health check** mechanism: the primary server verifies that the preferred server is alive and ready before advertising `preferred_address` in the handshake. If the preferred server is unhealthy, the primary simply serves the client directly without migration -- the client never knows migration was an option.

Result: Zero failed migrations. If the preferred server is down, clients are served directly by the primary with no timeout or error.

2. Architecture

2.1 Testbed

The PoC uses the same four-machine LAN testbed as our existing migration implementation:

Role	Machine	IP Address	Ports	Function
Client	optiplex7010	141.217.168.127	--	Firefox browser (HTTP/3)
Primary Server	opti7040	141.217.168.152	4433 (QUIC)	TLS handshake, health checks
Preferred Server	homeserver2	141.217.168.143	4433 (QUIC) 9998 (health) 9999 (state)	Imports state, handles <code>PATH_CHALLENGE</code>
Redis Server	Proxmox VM	141.217.168.200	6379 (Redis)	Heartbeat key storage (Redis mode)

2.2 Port Separation: Health (9998) vs State (9999)

A critical design decision in this PoC is the use of a **dedicated health check port (9998)**, fully separated from the state transfer port (9999). This separation was introduced to fix a serious bug in an earlier design:

Bug (fixed): The original TCP health probe connected to the state transfer port 9999. When the TCP probe connected and immediately disconnected (as probes do), the preferred server's TCP state

receiver would accept the empty connection and panic, because it expected to read 445 bytes of migration state but received zero bytes instead.

The fix is straightforward: the preferred server now listens on two separate TCP ports with distinct responsibilities:

Port	Purpose	Protocol	Behavior on Connect
9998	Health check endpoint	TCP connect-only (no data exchange)	Accepts connection, immediately closes. No data read/written.
9999	Migration state transfer	TCP push (445 bytes of TLS state)	Reads exactly 445 bytes, deserializes MigrationState, imports crypto.

This separation ensures that health probe connections can never interfere with the state transfer channel. The health listener is a trivial TCP acceptor that requires no data exchange -- a successful TCP three-way handshake is sufficient to prove the preferred server process is running and reachable.

2.3 Health Check Modes

The PoC supports two complementary health check strategies, each with different trade-offs in detection latency, infrastructure requirements, and failure modes:

TCP Probe

The primary server performs a TCP connect to the preferred server's **dedicated health port (9998)** with a **200ms timeout**. This is a per-connection probe: before each new QUIC handshake, the primary attempts a TCP connection to the preferred server. If the connection succeeds, the preferred server is considered healthy and preferred_address is advertised. If the connection fails or times out, the primary serves the client directly.

- Per-connection granularity -- every handshake is individually validated
- No external infrastructure required (no Redis dependency)
- Probes use port 9998 (health), never port 9999 (state transfer)
- Detection latency: bounded by the 200ms TCP timeout
- Trade-off: one extra TCP round-trip per connection (~0.1ms on LAN)

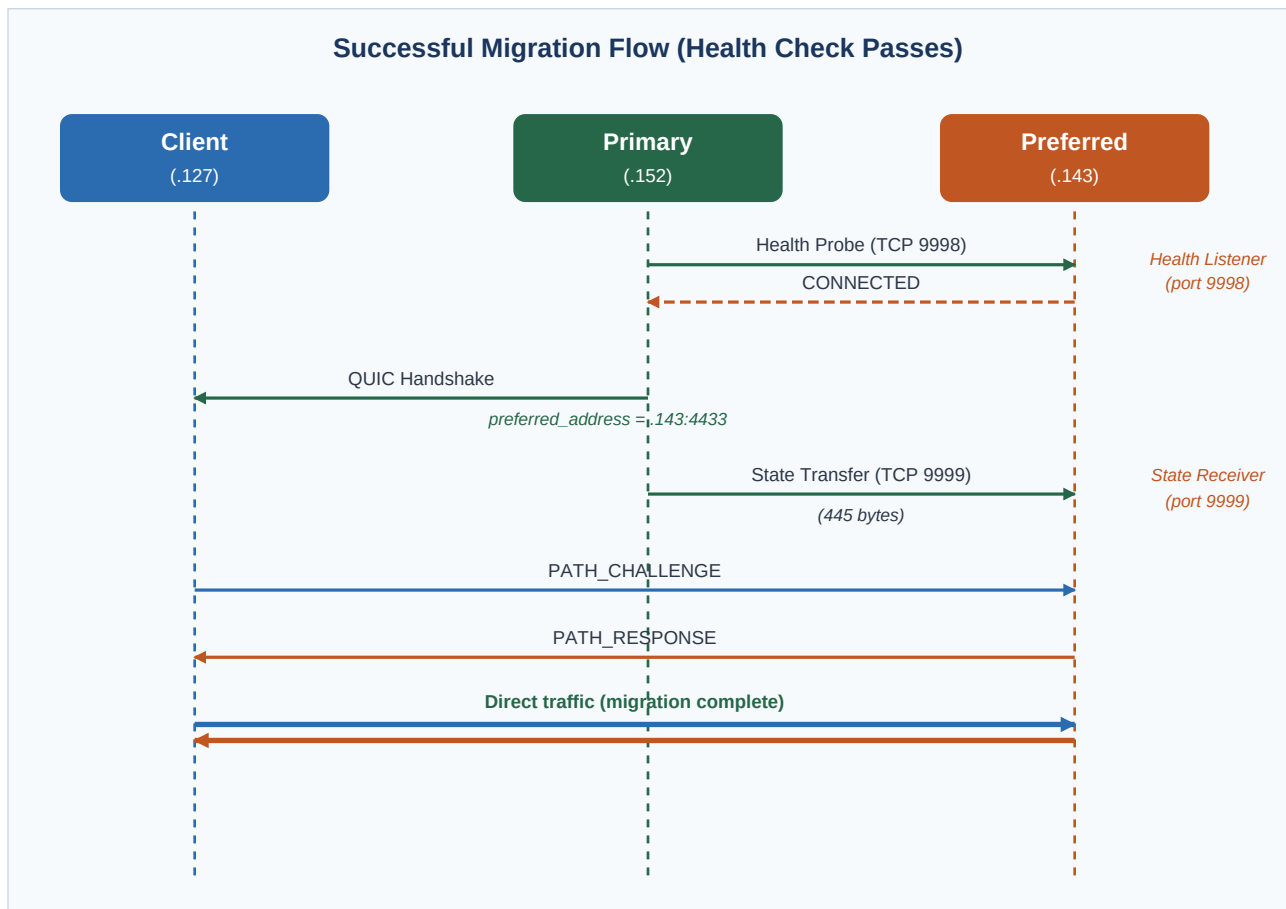
Redis Heartbeat

The preferred server publishes a preferred:health key to Redis every **1 second** with a **3-second TTL**. The primary server checks this key before each handshake. If the key exists, the preferred server has been alive within the last 3 seconds. If the key has expired (TTL elapsed), the preferred server is considered down.

- Decoupled: preferred server does not need to be directly reachable from primary
- Requires Redis infrastructure (already available on our testbed at .200)
- Detection latency: up to 3s (heartbeat TTL) after preferred server failure
- Trade-off: one Redis GET per connection (~0.1ms on LAN)

2.4 Migration Flow Diagram (Success)

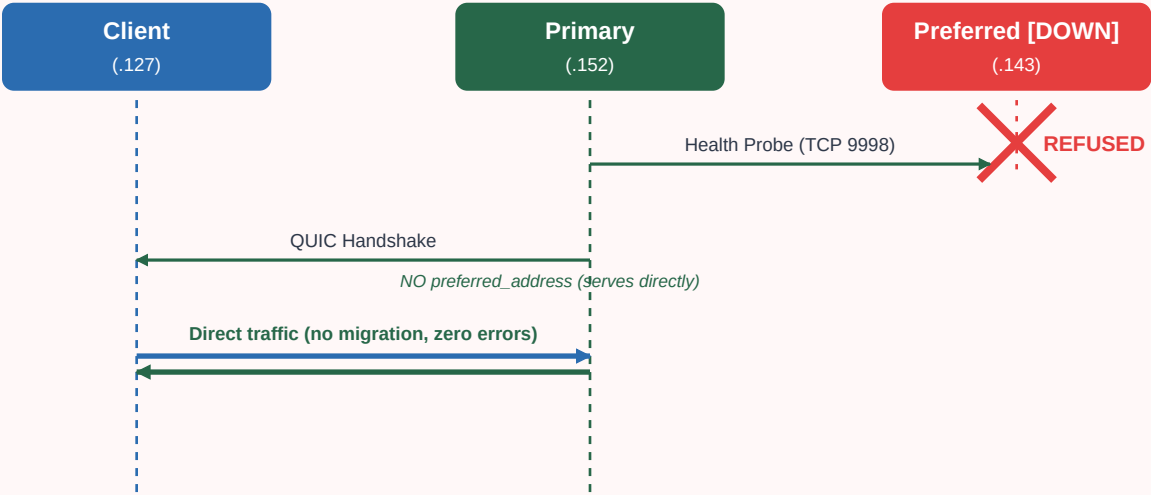
The following diagram shows the complete message flow when the health check passes and migration proceeds. Note the separation of health probe (port 9998) from state transfer (port 9999):



2.5 Failure Scenario Diagram

When the preferred server is down, the health probe to port 9998 fails (connection refused or timeout). The primary omits *preferred_address* from the handshake and serves the client directly. No state transfer occurs, no **PATH_CHALLENGE** is sent, and the client experiences zero errors:

Failure Scenario: Preferred Server Down



3. Implementation Details

3.1 Binaries and Configuration

The PoC introduces two new binaries in the `migration-test/` crate:

Binary	Description
health-check-primary	Modified primary server with health check logic. Conditionally advertises <code>preferred_address</code> based on health status. Probes port 9998 (not 9999).
health-check-preferred	Preferred server with health listener on port 9998 and state receiver on port 9999. In Redis mode, also publishes heartbeats.

Environment Variables

Variable	Values	Description
HEALTH_CHECK	tcp redis none	Selects the health check mode. Default: none (unconditional migration).
HEALTH_PORT	9998 (default)	TCP port for the dedicated health check listener on the preferred server. Must differ from the state transfer port.
REDIS_URL	redis://141.217.168.200:6379	Redis connection URL for heartbeat mode. Only used when <code>HEALTH_CHECK=redis</code> .

3.2 Background Health Thread

The primary server spawns a background thread that continuously monitors the preferred server's availability. This thread maintains a shared atomic boolean (`is_healthy`) that the main connection-handling code reads before each handshake.

3.3 Heartbeat Thread Behavior

The background heartbeat thread operates on a fixed **5-second polling interval**. Its behavior differs by mode but follows a common pattern:

TCP mode: Every 5 seconds, the thread attempts a TCP connect to the preferred server's health port (9998) with a 200ms timeout. If the connect succeeds, it sets `is_healthy = true`. If it fails (connection refused, timeout, or network error), it sets `is_healthy = false`.

Redis mode: Every 5 seconds, the thread performs a `GET preferred:health` against Redis. If the key exists (i.e., the preferred server has published within the TTL window), health is true. If the key is absent or Redis is unreachable, health is false.

In both modes, the thread **only logs when status changes**, avoiding log spam during steady state:

```
[health] preferred server status changed: HEALTHY -> UNHEALTHY
[health] preferred server status changed: UNHEALTHY -> HEALTHY
[health] re-check in 5s...
```

The 5-second interval is a balance between responsiveness and overhead. On a LAN, each probe costs ~0.1ms, so 5s polling adds negligible load. The worst-case detection latency is one full interval (5s) plus the probe timeout (200ms for TCP).

3.4 Redis Heartbeat Protocol

When using the Redis heartbeat mode, the preferred server communicates with Redis using the **RESP (Redis Serialization Protocol)**. The heartbeat is a single SET command with an expiry:

```
SET preferred:health "alive" EX 3
```

This command sets the key `preferred:health` to the value `"alive"` with a **3-second TTL** (the `EX 3` argument). The preferred server re-issues this command every 1 second, ensuring the key is always refreshed before it expires. If the preferred server crashes or becomes unreachable, the key expires after 3 seconds and the primary detects the absence on its next check.

RESP wire format for the SET command:

```
*5\r\n$3\r\nSET\r\n$16\r\npreferred:health\r\n$5\r\nalive\r\n$2\r\nEX\r\n$1\r\n3\r\n\r\n
```

The primary server reads the key with a simple `GET preferred:health`. A non-nil response means healthy; a nil response (key expired or never set) means unhealthy.

3.5 Conditional preferred_address

The key modification is in the connection setup path. Before constructing `ConnectionParameters`, the primary checks the health status:

```
if is_healthy.load(Ordering::Relaxed) {  
    // Set preferred_address in ConnectionParameters  
    // Export and transfer migration state after handshake  
} else {  
    // Serve client directly, no preferred_address  
    // Log: "preferred server unhealthy, serving directly"  
}
```

When healthy, the server behaves identically to the existing migration flow. When unhealthy, the server acts as a normal HTTP/3 server with no migration.

3.6 HTML Status Page

The HTTP/3 response page dynamically reflects the migration status. When the preferred server is healthy, the page shows a green "HEALTHY" badge and indicates that migration is active. When unhealthy, the page shows a red "UNHEALTHY" badge and states that the client is being served directly by the primary.

4. Security Considerations

The health check mechanism is designed to operate entirely outside the TLS security boundary. Several security properties are maintained by design:

Health probes do not expose TLS secrets

The TCP health probe on port 9998 performs a bare TCP connect with **no data exchange**. The probe establishes a TCP three-way handshake, then immediately closes. No TLS secrets, connection IDs, or migration state bytes traverse the health channel. The 445-byte migration state (containing TLS secrets and CIDs) only ever flows through the separate state transfer channel on port 9999.

Port separation prevents state corruption

Because health probes (port 9998) and state transfer (port 9999) use separate ports, a malicious actor probing port 9998 cannot inject data into the state transfer channel. Conversely, the state receiver on port 9999 never accepts probe-style empty connections that could cause deserialization panics.

Redis heartbeat security

In Redis heartbeat mode, the only data written to Redis is the string "alive" under the key `preferred:health`. No TLS key material, connection IDs, or client addresses are stored in Redis for health checking purposes. The Redis heartbeat is purely a liveness signal. If Redis is compromised, an attacker could forge or delete the heartbeat key, causing the primary to either always migrate or never migrate -- but could not intercept or modify TLS-encrypted QUIC traffic.

Fail-safe defaults

All failure modes default to **no migration**. If the health check infrastructure itself fails (Redis unreachable, TCP probe error, etc.), the primary serves clients directly. This means infrastructure failures cause loss of migration capability but never cause client-visible errors or security degradation.

5. Metrics and Monitoring

The health-check-primary server tracks and logs key migration metrics to enable operational monitoring and debugging:

Metric	Type	Description
migration_attempted	Counter	Total number of connections where the primary advertised preferred_address and attempted to transfer state.
migration_succeeded	Counter	Connections where the preferred server successfully received the migration state and the client completed PATH_CHALLENGE/PATH_RESPONSE.
migration_skipped	Counter	Connections where the health check failed and the primary served the client directly without advertising preferred_address.
health_status	Gauge (boolean)	Current health status of the preferred server as seen by the background thread. Logged on every status change.

Metric	Type	Description
health_check_latency	Duration	Time taken by each health probe (TCP connect or Redis GET). Typically <0.2ms on LAN.

These metrics are logged to stdout in a structured format. Example log output during normal operation:

```
[health] preferred server status: HEALTHY (probe latency: 0.08ms)
[conn] new connection from 141.217.168.127:52341
[conn] health check passed, advertising preferred_address
[conn] migration state exported (445 bytes), sending to .143:9999
[stats] attempted=14 succeeded=13 skipped=1
```

During a preferred server outage:

```
[health] preferred server status changed: HEALTHY -> UNHEALTHY
[conn] new connection from 141.217.168.127:52355
[conn] health check FAILED, serving directly (migration_skipped=2)
[stats] attempted=14 succeeded=13 skipped=2
```

6. How to Run

The PoC is launched from the same machines as the existing migration demo. The preferred server is started first, followed by the primary.

Step 1: Start the Preferred Server (homeserver2)

```
health-check-preferred 141.217.168.143:4433 9999 9998
```

This starts the preferred server listening on three ports: **4433** (QUIC traffic after migration), **9999** (TCP state transfer receiver), and **9998** (TCP health check listener). In Redis mode, it also begins publishing heartbeats to Redis every 1 second. The health listener on port 9998 is a minimal TCP acceptor that requires no data exchange -- it simply accepts and closes connections.

Step 2: Start the Primary Server (opti7040)

```
HEALTH_CHECK=tcp health-check-primary 0.0.0.0:4433 \  
141.217.168.143:4433 141.217.168.143:9999 141.217.168.143:9998
```

Arguments: (1) local bind address, (2) preferred server QUIC address, (3) preferred server state-transfer address (port 9999), (4) preferred server health-check address (port 9998). The HEALTH_CHECK variable selects the probe mode. The health probe always targets port 9998, never port 9999.

Step 3: Open Firefox

Navigate to <https://141.217.168.152:4433/>. The bootstrap server (Alt-Svc) must be running on opti7040 for the first HTTP/3 upgrade, same as the standard demo. After the first load, refresh to use HTTP/3 directly.

Tip: To switch between TCP and Redis modes, simply change the HEALTH_CHECK environment variable and restart the primary server. No changes needed on the preferred server side.

Important: The HEALTH_PORT environment variable (default 9998) can be used to customize the health check port. If changed, ensure both the preferred server and primary server use the same port value.

7. Test Scenarios

Baseline: Both Servers Up

1. Start both servers normally.
2. Load the page in Firefox -- migration succeeds as usual.
3. Expected: 100% successful migration, identical to existing demo.
4. Purpose: Confirms health check adds no regression.

Preferred Down: Kill Preferred Server

1. Start both servers, verify migration works.
2. Kill the preferred server (preferred-down on homeserver2).
3. Wait 5 seconds for the background health thread to detect the failure.

4. Load the page in Firefox again.
5. Expected: Primary detects unhealthy preferred, serves directly. Zero timeout, zero errors. Page loads normally.

Flapping: Start/Stop Preferred Every 5s

1. Start both servers.
2. In a loop, alternate: kill preferred, wait 5s, start preferred, wait 5s.
3. Load the page repeatedly during the flapping cycle.
4. Without health check: random PATH_CHALLENGE timeouts on ~50% of loads.
5. With health check: graceful fallback, zero timeouts.

Recovery: Preferred Comes Back Up

1. Start both servers, then kill the preferred server.
2. Verify primary serves directly (no migration).
3. Restart the preferred server.
4. Wait up to 5 seconds for the background health thread to detect recovery.
5. Load the page -- migration should resume.
6. Expected: Automatic recovery with no manual intervention.

8. Expected Results

The health check mechanism is expected to eliminate all user-visible failures during preferred server outages:

Without health check: When the preferred server is down, clients experience PATH_CHALLENGE timeouts of 100-600ms. In the worst case, the connection may fail entirely if the client's retry budget is exhausted.

With health check: Zero failed migrations. The primary detects the preferred server is down and serves clients directly. The transition is invisible to the client -- no timeout, no retry, no error.

Key Metrics

- **Failed migrations:** 0% (vs. 100% without health check when preferred is down)
- **Added latency (healthy path):** ~0.1ms per connection (TCP connect or Redis GET)
- **Detection latency (TCP):** <200ms (bounded by TCP timeout, per-connection)
- **Detection latency (Redis):** <3s (bounded by heartbeat TTL)
- **Recovery time:** <5s (background thread polling interval)
- **User-visible errors:** Zero during preferred server failures
- **Health port:** 9998 (dedicated, separate from state port 9999)

9. Comparison Table

The following table compares behavior across all scenarios with and without health checking enabled:

Scenario	Without Health Check	With Health Check (TCP)	With Health Check (Redis)
Preferred healthy	Migration OK	Migration OK	Migration OK
Preferred down	Client timeout (100-600ms)	Serves directly (probe to 9998 refused)	Serves directly (key expired)
Preferred flapping	Random timeouts	Graceful fallback	Graceful fallback
Detection latency	N/A (no detection)	<200ms (per-conn probe to 9998)	<3s (heartbeat TTL)
Added latency	0	~0.1ms (TCP connect to 9998)	~0.1ms (Redis GET)
Health port used	N/A	9998 (dedicated)	N/A (uses Redis)

Trade-off Summary

Aspect	TCP Probe	Redis Heartbeat
Infrastructure	None (direct TCP to port 9998)	Requires Redis server
Granularity	Per-connection	Periodic (1s heartbeat)
Detection speed	Immediate (<200ms)	Delayed (up to 3s TTL)
Health port	9998 (dedicated, no data exchange)	N/A (Redis key-based)
State port safety	9999 never touched by probes	9999 never touched by probes
Network topology	Primary must reach preferred port 9998	Both must reach Redis; no direct link needed
Failure mode	Probe fails = no migration (safe)	Redis down = key absent = no migration (safe)
TLS secret exposure	None (probe is data-free)	None (only "alive" string in Redis)
Best for	Low-latency, simple setups	Multi-datacenter, indirect routing

Both modes fail safe: if the health check itself fails (e.g., Redis is unreachable or port 9998 is blocked by a firewall), the primary defaults to serving directly without migration. This ensures that health check infrastructure failures never cause client-visible errors.